AI ML PROJECT REPORT

Sneha Jadhav
25BAI11265
FACULTY: Satendra Singh

Introduction

Artificial Intelligence and Machine Learning in Real Life

Artificial Intelligence (AI) and Machine Learning (ML) have become powerful tools that are quietly transforming our everyday lives. From recommending movies on Netflix to helping doctors detect diseases early, AI/ML is making things faster, smarter, and more personalized.

Key Ways AI/ML Improves Our Lives:

- **Healthcare**: Early disease detection, personalized treatment plans, and medical imaging analysis.
- **Daily Convenience**: Smart assistants (like Siri, Alexa), traffic prediction, and personalized shopping recommendations.
- **Education**: Adaptive learning platforms that tailor lessons to individual student needs.
- **Safety & Efficiency**: Fraud detection in banking, autonomous vehicles, and predictive maintenance.

By automating repetitive tasks and uncovering hidden patterns in data, AI/ML helps us work smarter, save time, and lead healthier, more convenient lives.

Problem Statement

Phishing attacks remain one of the most common and dangerous cyber threats worldwide. Fraudsters send fake emails and messages that appear legitimate to steal personal information, passwords, financial details, and even install malware. Despite growing digital communication, most users cannot easily identify sophisticated phishing attempts. This results in huge financial losses, identity theft, and data breaches every year. Lack of awareness and absence of simple, real-time detection tools make ordinary people highly vulnerable.

Scope of the Project

The website will focus on:

- Real-time detection of phishing in emails and text messages (SMS/WhatsApp-style)
- AI/ML-based analysis with clear explanations of suspicious elements
- Built-in awareness and education modules for users
- User-friendly interface for non-technical people

Requirements

- Functional: Upload/paste email or message → instant phishing score + highlighted red flags → awareness tips
- Educational section with quizzes, real examples, and prevention guides
- User dashboard to save scan history
- Responsive, secure, and fast web platform
- Basic admin panel for updating detection rules

System Architecture

```
[ User Browser (Mobile/Desktop) ]
          ↓ (HTTPS)
[ Frontend Layer ]  ↔  [ Backend Layer (Python) ]  ↔  [ ML Detection Engine ]
          ↓                                       ↓
     Static Files + JS                      [ Database Layer ]
          ↓                                       ↓
     Awareness Content                     Awareness Knowledge Base
```

Layer	Component	Python Tech / Tools Used	Responsibility
Frontend	UI/UX	HTML5 + Bootstrap + JavaScript (Vanilla)	Input form, results display, quiz, dashboard
Backend	Web Framework	**Flask** (lightweight) or Django	Routing, authentication, API endpoints
AI/ML Engine	Phishing Detector	scikit-learn / TensorFlow / Hugging Face	Text classification, URL analysis, score (0-100)
Data Layer	Database	SQLite (dev) / PostgreSQL (production)	Store scan history, user data, awareness articles
Knowledge Base	Awareness Module	Flask + Jinja2 templates	Articles, quizzes, tips, real examples
Utilities	Email/Message Parser	Python `email` + regex + BeautifulSoup	Extract links, sender, subject, body

```
phishing-awareness/
├── app.py                  # Main Flask app
├── models/                 # ML model files (phishing_model.pkl)
├── controllers/
│   ├── detection.py
│   └── awareness.py
├── templates/              # HTML pages
├── static/                 # CSS, JS, images
├── database/               # schema + migrations
├── utils/                  # text parser, feature extractor
└── knowledge_base/         # JSON/articles for awareness
```

TESTING APPROACH

The testing strategy for the Phishing Detection and Awareness Website follows a structured, multi-level approach to ensure accuracy, reliability, security, and user-friendliness. Both functional and non-functional aspects will be thoroughly validated.

1. Testing Levels

- Unit Testing
  - Test individual components such as text preprocessing, feature extraction, and ML model prediction.
  - Tools: pytest and unittest (Python)
- Integration Testing
  - Verify interaction between Frontend, Backend (Flask), ML Engine, and Database.
  - Test end-to-end flow: message upload → detection → result display.
- System Testing
  - Test the complete website as a whole including detection accuracy, awareness module, and user dashboard.
- User Acceptance Testing (UAT)
  - Conducted with real users (students/teachers) to validate ease of use and effectiveness of awareness content.

2. Testing Types

- Functional Testing
  - Phishing detection accuracy (score generation, red flag highlighting)
  - Awareness features (quizzes, articles, tips)

    - User registration, login, and scan history
- Non-Functional Testing
    - Performance: Response time under normal and peak load
    - Usability: Responsive design on mobile/desktop
    - Security: Input validation, SQL injection prevention, XSS protection
    - Reliability: Model consistency with different phishing samples
- ML-Specific Testing
    - Accuracy, Precision, Recall, and F1-score on test dataset
    - False Positive and False Negative rate analysis
    - Testing with real-world phishing and legitimate samples

3. Tools & Techniques

- Python: pytest, unittest
- Manual testing for UI/UX
- Test datasets: Public phishing datasets + custom collected samples
- Browser testing: Chrome, Firefox, Mobile browsers

Expected Outcomes:

High detection accuracy (>85%), zero critical security bugs, and positive user feedback on awareness modules.

DESIGN DAIGRAM

Workflow Diagram (Main Detection Flow)

```text
Start
  ↓
User Selects Module (URL / Email)
  ↓
Input Data (URL or Email Text)
  ↓
Preprocessing (Clean text, extract features)
  ↓
Heuristic Analysis
   ├── URL Analysis (Homoglyph, IP, TLD, Keywords, etc.)
   └── Email Analysis (Urgency, Harvesting, Greeting, etc.)
  ↓
Calculate Risk/Threat Score (0-100)
  ↓
Generate Alerts (High/Med/Low) + Color-coded Verdict
  ↓
Display Results (Neon HUD + Progress Bar + Highlights)
  ↓
Offer Download Report + Awareness Tips
  ↓
Update XP & Rank
  ↓
End (Option to scan again or go to Training)
```

Sequence Diagram (URL Scan Example)

```text
User          Streamlit App          analyze_url() Function
 |                  |                        |
 |---Enter URL------>|                        |
 |                  |---Call analyze_url()--->|
 |                  |                        | (Check Punycode, IP, TLD,
 |                  |<--Return score + flags--|
 |                  |                        |
 |<--Show Score + Verdict + Alerts---------|
 |                  |                        |
 |---Click Download->|                        |
 |<--Generate .txt Report-----------------|
```

Class/Component Diagram (Simplified)

Main Components:

- PhishGuardApp (Streamlit Main)
  - Manages session_state (xp, active_tab, quiz_score)
  - Renders UI (Navigation, Tactical Cards, Neon CSS)
- Analyzer Module
  - analyze_url(url) → Returns (score, flags)
  - analyze_email(text) → Returns (score, alerts)
- Gamification Engine
  - update_xp(amount)
  - get_rank() → Based on XP thresholds
- Report Generator
  - Creates timestamped .txt reports with findings

Key Interactions:

- Streamlit UI → Calls Analyzer → Updates Gamification → Displays Results

Design Decisions & Rationale

1. Streamlit Framework
    - Why? Extremely fast prototyping, built-in state management, and excellent for data/AI demo apps. No need for a separate frontend/backend. Ideal for educational + detection tools.
2. Rule-based Heuristic Engine (instead of full ML model)
    - Why? Lightweight, explainable, and fast. No dependency on heavy ML libraries. Easy to understand red flags (which helps awareness goals). Can be upgraded later to scikit-learn or Hugging Face.
3. Cyberpunk / Tactical Neon UI Theme
    - Why? Makes the tool engaging and memorable. Gamification (XP + Ranks) encourages repeated use and learning. Users feel like "cyber operators" while learning phishing defense.
4. No Sidebar + Top Navigation
    - Why? Cleaner immersive experience. All modules accessible in one click. Hides default Streamlit elements for professional "HUD" feel.
5. XP & Rank System
    - Why? Adds gamification to spread awareness. Users get rewarded for scanning and training, motivating them to learn prevention techniques.
6. Downloadable Reports
    - Why? Allows users to save evidence for personal records or reporting to IT/security teams.

Strengths

- Highly visual and interactive
- Strong focus on awareness + detection
- Easy to run locally (streamlit run app.py)

Future Improvements

- Integrate real ML model (scikit-learn/TF) for better accuracy
- Add user login & history database
- URL live checking (requests + WHOIS)
- More quiz questions and phishing examples

Challenges Faced

Developing the Phish-Guard phishing detection and awareness website presented several technical and design challenges:

- Limited Detection Accuracy with Rule-Based Heuristics — The current system relies on static rules for URL and email analysis. It struggles with sophisticated, zero-day phishing attacks, new evasion techniques (e.g., subtle typo-squatting, AI-generated content), and can produce false positives/negatives.
- Absence of Real-Time External Validation — No live URL checking (WHOIS, domain age, SSL verification) or integration with external APIs due to simplicity and deployment constraints.
- Balancing UI/UX with Performance — Creating an immersive cyberpunk neon "tactical HUD" while keeping the app lightweight and responsive in Streamlit was challenging. Heavy custom CSS sometimes affected rendering speed.
- Gamification vs Educational Depth — Designing an engaging XP/Rank system without making the awareness module feel like a game at the expense of serious learning.
- Scope Management — Starting with a full ML model was avoided for faster prototyping, but it limited accuracy compared to trained classifiers.
- Deployment & Security — Ensuring safe handling of user-input data (preventing injection risks) while maintaining a guest-friendly interface.

Key Learnings

- Rule-based systems are fast, highly explainable, and ideal for educational tools, but they lack adaptability. Machine Learning models (e.g., Random Forest, XGBoost) offer better generalization for phishing detection.
- Gamification (XP, ranks, neon visuals) significantly increases user engagement and motivates repeated use of awareness features like quizzes.
- Streamlit is excellent for rapid prototyping of interactive AI/ML demos, allowing focus on core logic rather than full-stack development.
- Effective phishing awareness requires clear, highlighted "red flags" with actionable tips rather than just a binary safe/unsafe result.
- Feature engineering (keywords, urgency tactics, URL patterns) is critical — even simple heuristics can teach users valuable prevention skills.

Takeaways

- Hybrid Approach is Best: Combine rule-based detection for transparency + ML for accuracy in future versions.
- User Education Matters Most: Detection tools alone are insufficient; interactive training and real examples reduce click rates more effectively.
- Explainability Wins Trust: Showing why a message/URL is flagged builds user confidence and learning.
- Make Security Engaging: Cyber-themed UI and rewards turn dry awareness training into an immersive experience.
- Start Simple, Scale Later: The current MVP successfully demonstrates core functionality and can be upgraded with datasets, ML models, and database support.

References

1. Streamlit Documentation – Official guide for building interactive web apps.
2. Phishing URL Detection using Machine Learning (Various studies on heuristic vs ML approaches).
3. "A Systematic Literature Review on Phishing Website Detection" – Highlights limitations of rule-based methods.
4. Research papers on URL feature extraction and heuristic phishing detection (IRJET, ResearchGate publications).

5. Best practices from Kaggle notebooks and GitHub repositories on phishing detection dashboards.